

Red de Distribución

Autor: Randy Valle López

Curso: Ingeniería Informática 2do, Curso 25-26

Problemática y Objetivos

El presente proyecto surge de la necesidad de dar solución a la dificultad que representa la planificación y gestión eficiente de una red de distribución compuesta por múltiples almacenes, buscando minimizar los costos asociados al transporte y optimizar las rutas utilizadas para la distribución de mercancías.

El objetivo principal de este proyecto es reducir los costos de transporte dentro de una red de distribución mediante la aplicación de estructuras de datos como grafos y árboles. Estas estructuras resultan especialmente adecuadas para modelar este tipo de problemas, ya que un grafo permite representar de forma precisa una red de almacenes y las conexiones existentes entre ellos, además de proporcionar algoritmos capaces de determinar rutas de costo mínimo entre distintos puntos de la red. Por otra parte, se emplea un árbol general para representar de manera jerárquica la red mínima de distribución obtenida a partir del grafo. Esta representación facilita la visualización de los almacenes que conforman la red, así como la comprensión de las relaciones existentes entre ellos, permitiendo una interpretación más clara de la estructura de distribución generada

Diseño del Sistema

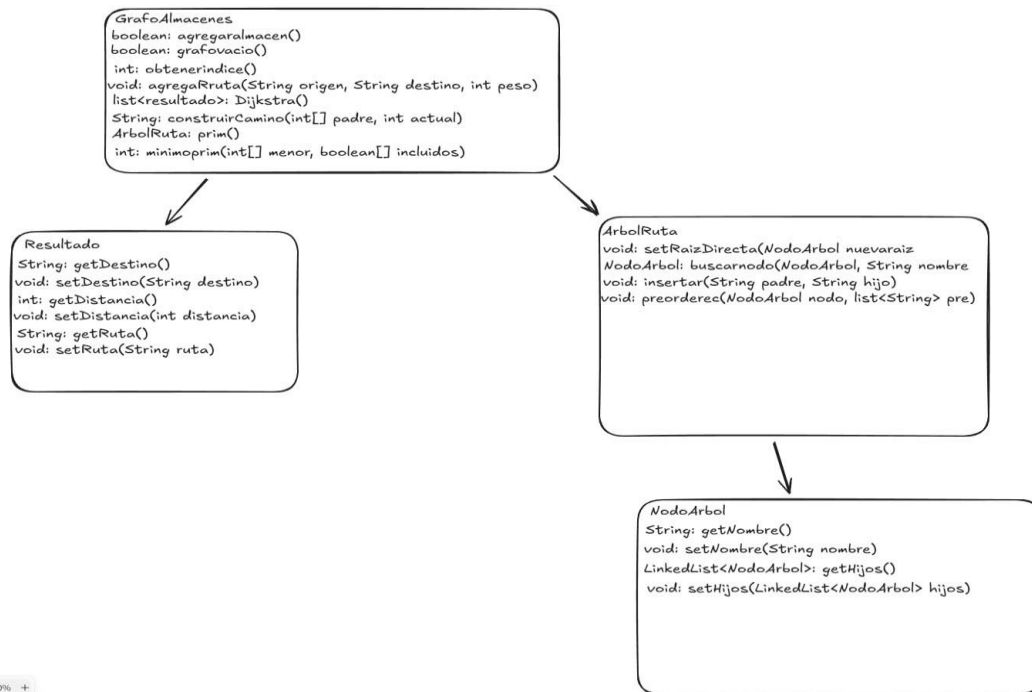
El diseño de este proyecto permite una integración coherente entre las estructuras de datos empleadas, aprovechando las ventajas que cada una ofrece para la resolución del problema planteado.

En primer lugar, el grafo constituye la estructura principal del sistema, ya que permite representar de forma adecuada una red de almacenes y las conexiones existentes entre ellos. Cada vértice representa un almacén, mientras que las aristas representan las rutas de comunicación o

transporte, asociadas a un costo o distancia determinada. A partir de esta representación, se implementan algoritmos que permiten analizar y optimizar la red.

Entre ellos destaca el algoritmo de Prim, cuya función es obtener un árbol de expansión mínima a partir del grafo. Este árbol garantiza la conexión de todos los almacenes de la red utilizando el menor costo total posible, eliminando conexiones redundantes y optimizando la infraestructura de distribución. El resultado obtenido mediante el algoritmo de Prim es posteriormente utilizado para construir y rellenar un árbol general. Esta estructura permite representar de manera jerárquica la red generada, facilitando su organización y visualización. Gracias a ello, es posible recorrer la red de forma ordenada mediante recorridos como el preorden, mostrando todos los almacenes que forman parte de la red de distribución y permitiendo una interpretación más clara de las conexiones establecidas entre ellos.

Diagrama de clases



Algoritmo de Dijkstra

```

public List<resultado> dijkstra(String origen) {
    int origenI = obtenerIndice(origen);    if (origenI
    == -1) {
        return new ArrayList<>();
    }
    int[] distancias = new int[cantidadVertices];
    boolean[] visitados = new boolean[cantidadVertices];
    int[] padres = new int[cantidadVertices];
    for (int i = 0; i < cantidadVertices; i++)
    {
        distancias[i] = Integer.MAX_VALUE;
        visitados[i] = false;
        padres[i] = -1;
    }
    distancias[origenI] = 0;
    for (int i = 0; i < cantidadVertices - 1; i++) {
        int u = VerticeMinimo(distancias, visitados);
        if (u == -1) {
            break;
        }
        visitados[u] = true;
        for (int v = 0; v < cantidadVertices; v++) {
            if (
                !visitados[v]
                && matriz[u][v] != 0

                && distancias[u] != Integer.MAX_VALUE
                && distancias[u] + matriz[u][v] < distancias[v]
            )
        }
    }
}
  
```

```

        ) {
            distancias[v] =
                distancias[u] + matriz[u][v];

            padres[v] = u;
        }
    }
}

```

Este fragmento de código pertenece al algoritmo Dijkstra. El algoritmo de Dijkstra fue implementado para determinar la ruta de menor costo entre un almacén de origen y el resto de los almacenes de la red. Para ello, se utiliza la matriz de adyacencia del grafo, donde cada posición almacena el costo asociado a una conexión entre dos almacenes.

Durante la ejecución del algoritmo se emplean arreglos auxiliares para almacenar la información necesaria. El arreglo de distancias mantiene el costo mínimo conocido desde el almacén de origen hasta cada almacén de la red, mientras que el arreglo de padres registra el almacén previo en la ruta óptima encontrada. Adicionalmente, se utiliza un arreglo de vértices visitados para evitar procesar repetidamente almacenes cuya distancia mínima ya ha sido determinada.

El algoritmo comienza asignando una distancia inicial infinita a todos los almacenes, excepto al almacén de origen, cuya distancia se establece en cero.

Posteriormente, en cada iteración se selecciona el almacén no visitado con la menor distancia acumulada y se examinan sus almacenes vecinos. Si se encuentra una ruta con un costo inferior al registrado previamente, se actualizan tanto la distancia como el padre correspondiente.

Algoritmo de Prim

```

public ArbolRuta prim() {
    int[] padres = new int[cantidadVertices];
    int[] menor = new int[cantidadVertices];
    boolean[] incluidos = new boolean[cantidadVertices];

    for (int i = 0; i < cantidadVertices; i++) {
        menor[i] = Integer.MAX_VALUE;
        incluidos[i] = false;
        padres[i] = -1;
    }
}

```

```

    }
    menor[0] = 0;
    for (int i = 0; i < cantidadVertices - 1; i++) {
int u = MinimoPrim(menor, incluidos);          if (u == -1) {
break;
    }
    incluidos[u] = true;
    for (int v = 0; v < cantidadVertices; v++) {
        if (matriz[u][v] != 0 && !incluidos[v] && matriz[u][v] < menor[v])
    {
        menor[v] = matriz[u][v];
padres[v] = u;
    }
    }
    }

    NodoArbol[] nodos = new NodoArbol[cantidadVertices];    for
(int i = 0; i < cantidadVertices; i++) {                nodos[i] = new
NodoArbol(almacenes[i]);
    }
    for (int i = 1; i < cantidadVertices; i++) {
int padre = padres[i];          if (padre != -1) {
        nodos[padre].getHijos().add(nodos[i]);
    }
    }

    ArbolRuta arbol = new ArbolRuta(almacenes[0]);
arbol.setRaizDirecta(nodos[0]);    return arbol;
}

```

El algoritmo de Prim fue implementado con el objetivo de obtener una red mínima de distribución que conecte todos los almacenes utilizando el menor costo total posible. Para ello, se parte de la representación de la red mediante una matriz de adyacencia, donde cada posición almacena el costo asociado a la conexión entre dos almacenes.

Durante la ejecución del algoritmo se emplean tres arreglos auxiliares: *padres*, *menor* e *incluidos*. El arreglo *menor* almacena el costo mínimo conocido para incorporar cada almacén al árbol de expansión mínima, mientras que *padres* registra el almacén desde el cual se realiza dicha conexión. Por su parte, el arreglo *incluidos* permite identificar qué almacenes ya forman parte de la red mínima construida hasta el momento.

Inicialmente, todos los almacenes reciben un costo infinito, excepto el almacén de origen, cuyo costo se establece en cero para iniciar la construcción del árbol. En

cada iteración se selecciona, mediante el método auxiliar `MinimoPrim`, el almacén no incluido con el menor costo asociado. Una vez seleccionado, dicho almacén se marca como incluido y se examinan sus almacenes vecinos para determinar si existe una conexión de menor costo que la registrada previamente. En caso afirmativo, se actualizan los arreglos menor y padres.

Al finalizar el proceso, el arreglo `padres` contiene la información necesaria para reconstruir el árbol de expansión mínima. A partir de estos datos se crea un arreglo de objetos `NodoArbol`, donde cada posición representa un almacén de la red. Posteriormente, utilizando la información almacenada en `padres`, se establecen las relaciones padre-hijo entre los nodos correspondientes, construyendo así una representación jerárquica de la red mínima obtenida.

Finalmente, el nodo principal de la estructura se asigna como raíz del árbol general mediante el método `setRaizDirecta`, obteniéndose una estructura organizada que permite visualizar y recorrer todos los almacenes pertenecientes a la red de distribución.

Recorrido en preorden

```
private void preordenrec(NodoArbol nodo, List<String> pre){    if
(nodo == null) return;

    pre.add(nodo.getNombre());

    for (NodoArbol hijo : nodo.getHijos()){                preordenrec(hijo,
pre);
    } }
```

El recorrido en preorden se utiliza para visitar todos los nodos del árbol general siguiendo un orden específico: primero se procesa el nodo actual y posteriormente se recorren de forma recursiva todos sus hijos. En el contexto de este proyecto, este recorrido permite visualizar todos los almacenes que forman parte de la red mínima de distribución obtenida mediante el algoritmo de Prim.

La implementación se realiza mediante el método recursivo `preordenrec`, el cual recibe como parámetros el nodo actual que se está procesando y una lista donde se almacenará el resultado del recorrido. Inicialmente, se verifica si el nodo

recibido es nulo. En caso de serlo, el método finaliza su ejecución, constituyendo el caso base de la recursión.

Si el nodo existe, su nombre se agrega a la lista mediante la instrucción `pre.add(nodo.getNombre())`, registrando así la visita al almacén correspondiente. Posteriormente, se recorre la colección de hijos asociada al nodo actual mediante un ciclo `for`, invocando nuevamente el método `preordenrec` para cada uno de ellos.

Gracias a este proceso recursivo, todos los almacenes pertenecientes al árbol son visitados exactamente una vez, respetando la estructura jerárquica definida en la red mínima de distribución. El resultado final es una secuencia ordenada de almacenes que permite visualizar la composición completa de la red a partir del almacén principal.

Casos de prueba

Se utilizó una red compuesta por seis almacenes conectados mediante rutas con diferentes costos. A partir del almacén Central como origen, se ejecutó el algoritmo de Dijkstra para obtener la ruta de menor costo hacia cada uno de los demás almacenes de la red. La salida esperada consistió en mostrar la distancia mínima y el camino óptimo correspondiente para cada destino.

Origen

Central

Ejecutar

Salir

Destino	Distancia	Ruta
Central	0	Central
Norte	4	Central -> Norte
Sur	2	Central -> Sur
Este	3	Central -> Sur -> Este
Oeste	7	Central -> Sur -> Oeste
Puerto	5	Central -> Sur -> Este -> Puerto

Sobre la misma red se ejecutó el algoritmo de Prim para generar la red mínima de distribución. Posteriormente, se realizó un recorrido en preorden sobre el árbol general construido a partir de dicha red. La salida esperada fue la visualización de todos los almacenes pertenecientes a la red mínima de distribución en el orden determinado por el recorrido:

Central, Sur, Este, Norte, Puerto y Oeste.

Central

Sur

Este

Norte

Puerto

Oeste

Conclusiones

El desarrollo de este proyecto permitió aplicar de manera práctica estructuras de datos fundamentales como grafos y árboles generales para la resolución de un problema relacionado con la gestión de redes de distribución. Mediante la implementación de los algoritmos de Dijkstra y Prim, fue posible optimizar las rutas de transporte y generar una red de distribución que conecta todos los almacenes con el menor costo posible.

La principal dificultad encontrada durante el desarrollo estuvo relacionada con la construcción del árbol general a partir de la información generada por el algoritmo de Prim. Fue necesario analizar cuidadosamente la relación entre los nodos y sus padres para lograr representar correctamente el árbol de expansión mínima dentro de una estructura jerárquica que permitiera su posterior recorrido y visualización.

Finalmente, el proyecto permitió comprender la importancia de seleccionar estructuras de datos adecuadas para cada problema y evidenció cómo diferentes estructuras pueden complementarse entre sí para obtener soluciones más completas y eficientes. Además, contribuyó al fortalecimiento de los conocimientos relacionados con algoritmos de grafos, árboles y técnicas de programación orientada a objetos.

Bibliografía

Se utilizó la IA ChatGPT para poder generar las clases de la interfaz de este proyecto